Computer Games Development
Project Report
Year IV

Olawole Abayomi-Owodunni
C00278711
13th of April 2026

[Declaration form to be attached]

Contents

Acknowledgements 3
Project Abstract 3
Project Introduction and/or Research Question 3
Literature Review 4
Evaluation and Discussion 5
Conclusions 8
References 8
Appendices 8

Acknowledgements

I would like to thank the following people who assisted in completing this project including;

Georgi Gerganov, llama.cpp creator

Ryan C. Holloway of SETU Carlow, Computer Games Development Year 4 for playtesting the project.

Project Abstract

Large Language Models have been in use in the industry for the better part of 5 years so far. We've seen its applications in various fields such as Software engineering, Health care, Finance, etc. Gaming however, is a field that remains untapped, only being used in aiding its development.

"In gaming, LLMs generate dynamic narratives, dialogues, and character interactions. This enhances the gaming experience by creating more immersive and responsive game worlds (GeeksforGeeks, 2025)." This shows that LLMs are very useful as tools to developers and whilst as game developers, we need to seek out ways to streamline development for the benefit of the players we should also be curious in how we can use this technology to enhance the experience of the Player.

I spent my time at work experience last year as a part of Arm Holdings Strategy and Ecosystems ML team. The team was working on a Real Time Voice Assistant application and within it was a submodule for a Large Language Models repository. The team was essentially bringing local inference to an edge device with incredible performance considering the compute power of the chip. This inspired me to bring this technology to gaming space, attempting to use local LLM inference technology to create avenues to enhance the collective experience of the player.

Project Introduction and/or Research Question

In 2014, Monolith productions released Middle Earth shadow of Mordor and with it, the nemesis system. This one system and idea created one of the most unique mechanics in gaming ever but unfortunately in February 2021, parent company Warner Bros patented the system with an expiry not until 2036. LLM Technology is the closest thing to rival the potential of the nemesis system in video games, and we are yet to see it's wiring into many of the video games we play today.

The purpose of this project is to explore if and how utilising LLM inferencing in games can lead to a more engaged user experience. If successful, could lead to the development of games that could revolutionise how gamers consume video games today and introduce a whole new world of immersion.

Does the inclusion of llm technology in of itself drive interaction and can this interaction increase player immersion? That is the question.

To test this, we need 2 deliverables:

1. An LLM Wrapper that uses a backend inference system like llama.cpp and streams the tokens generated from the prompt provided from the backend, through the custom wrapper, into the vendored llm system created within an SFML project.
2. An SFML project that can take advantage of the technology provided in by enabling the player to interact with NPC's like they would a chatbot and then using the custom library to generate in game lore based off game context.

However, before we dive deep, let us first explore some key concepts in Large Language Model technology.

For getting local llm inference working in our game, creating my own backend would have been a task too complicated and beyond the scope for the goal of this project and so instead of building a backend from scratch, I decided to go with using an already existing back end creating a custom C++ wrapper to take those wrapper functions and pipe it into my llm library, ready to do some processing. The process of getting our generated tokens is usually slow though especially with systems of lower power

It's important to note that typically you, the end user, never get access to a full model. They are broken down into parameters. The less parameters the model has, the less "powerful" it is. This power however comes at the cost of system resources, and this is why data centres that run full models require all the resources they end up consuming. Our memory space is limited but we still want to get good results regarding our inference. This is where quantisation comes in. It is the process of reducing the models' weights and activations (weights are the dials that determine its accuracy). This sacrifices accuracy for a great boost in speed and resources consumed.

For my project, I decided to go with a Llama-3.2-1B-instruct model quantised to q4_k_m format due to its retained accuracy and much less file size than other. Llama.cpp is an open-source backend provided to us by Georgi Gerganov (ggerganov) and this is the backend I chose to go with due to my familiarity with it for other projects.

The SFML wrapper is, on a high-level, a simple custom API. It has been designed as a Lego piece that can fit on top of any C++ SFML project by just creating a function within you sfml game code that can output a string, making it very modular and reusable in different areas. It also leaves it open for extension, should anyone want to try their hand on implementing this within a project of their own.

The world is moving to doing local inference on edge devices like the new Samsung galaxy devices and I would say it is important to keep on top of this trend by trying my hand at local inference in my field.

Literature Review

Large language models (LLMs) are artificial intelligence systems trained on vast amounts of data that can understand and generate human language (eweek, 2024). The core discovery that brought this technology to the forefront was Google's development of Transformer architecture in 2017 (fisclouds, 2024). To understand why the development of this technology

revolutionised Machine Learning, we must look at how ML algorithms worked prior to Transformers and to understand that we need to ask the question, what is a token?

In the ML space, a token is a fundamental unit of data; a word, part of a word, character, or punctuation mark used by language models to process, understand, and generate text (Nvidia, 2025). Prior to transformers deep learning models were dependent on Recurrent Neural Networks (RNN) and Long Short-Term Memory networks (LSTMs). These models process input token-by-token and that is incredibly slow. You can think of a transformer kind of like setting up another thread to run a process on in programming. Instead of running the through the input token-by-token, transformers process sequences in parallel, essentially allowing them to see the entire sequence at once making it a lot more efficient at focusing on the right tokens to grab not just everything (Medium, 2025).

Now let's talk about inference and tokenisation. Inference is the process of using already trained models to make predictions or decisions based on new data. This could be seen as a form of supervised learning since the dataset is provided to the model and then the model uses its data set to make an educated guess on what the input's output should be.

Tokenisation is the process of breaking down raw text into tokens. The transformer does its processing using tokens and tokenisation is our way of getting the output tokens from the transformers in a form that we can understand. How does this tie back to inference? Inference predicts what the next token produced should be. Tokenisation takes, that and gives it to us in a format that we can understand.

Those are the key technical terms in a nutshell.

You tend to see this sort of technology in the greater software world all the time but it is very rare to find this sort of technology utilised within games. So it was much to my surprise that I discovered a game built using C++ and SFML utilising llama.cpp for local inference.

On April 9, 2024 GitHub user MorganR08 released Lucys Labrynth, a puzzle game where the player plays as a girl in a maze of truth and lies. He was very clever here to use the model here as a gate. The user encounters doors throughout the labyrinth (with AI for brains), each of which has a profession, and either a preference for lying, or honesty. They ask the AI door any question they type, and it will answer. You get it right, you move on. You fail, game over (GitHub, 2024). He achieved this by cloning down the llama.cpp repo and piping the backend into his frontend directly using a Mistral 7b instruct model. I was relatively confident that integrating this was possible prior to witnessing this project discovering it was boost to prove the concept can be achieved.

Evaluation and Discussion

Getting the model to run with the wrapper was a huge relief to get working but it was running incredibly slow. Part of this was due to my configuration of the wrapper and my decision to not use certain features to preserve the structure of the backend. KV_Cache which stores the context of previous conversations for example is one of these things. After conducting a bit more research, I found that llama.cpp when running on x86 architectures struggles heavily because of the access to less memory space. After building for x64 release, I discovered that

not only saw x64 providing almost 10x the speed as x86 but the that the release builds for both x74 and x86 were significantly faster.

My findings can be viewed in this table:

Build	Bootup (model load)	Inference samples	Average inference time
x64 Debug	15.46 s	00:30.23, 00:29.21, 00:30.66	00:30.03
x64 Release	2.23 s	00:05.51, 00:05.38, 00:05.58	00:05.49
x86 Debug	33.12 s	1:07.11, 1:08.64, 1:08.69	1:08.15
x86 Release	1.25 s	00:10.87, 00:09.70, 00:09.32	00:09.96

This graph just shows how much more optimised llama is when using the extra memory space allowed because of the bigger buffer size.

Another serious point for speed was switching to using asynchronous generation by running the generation on a separate worker thread to keep the entire program from stalling whenever a new request for some tokens was made. Boot times became near instant, and inference was further improved by another 50 %.

Project Milestones

There are 4 main milestones that marked significant points at me getting closer to answering the question for my project.

The first major milestone was getting the Wrapper builds with no errors. This was achieved on the 3rd of April 2026. It took days of parsing through CMake code and understanding the intricacies of llama's functions and how to properly pull tokens to get this to work. I built a small test app which just feeds it a prompt using the wrapper and then uses the wrapper to receive its response. With this there was renewed confidence in pushing to get the LLM integrated into the game

The second was completing the Dungeon system which marked a significant step in getting the base game that the wrapper would sit on ready for integration. This was completed on the 4th of April 2026, very late in development but a needed feature to ensure the completion of the project. The dungeon works via a 3-layer system of rooms, floors and dungeons. Rooms contain the individual information on its function and contains all the meta data for the things to happen in said room. This system was so flexible that I used it to create the hub world that the player spawns in and out of between runs. The Floors, which build a contain a link of

nodes (rooms) and are deterministically (random but with a known seed) generated. The ownership goes: Dungeon -> Floor -> Room.

The 3rd major milestone was successfully integrating the wrapper into the game and getting it load return information about the room the player was currently in. This was accomplished on the 8th of April 2026, shortly before the due date.

The final major milestone was getting the LLM Dialogue system working. Configuring the prompt with the meta data of the NPC's and seeing the NPCs response based off that proved that there was promise in utilizing this technology in a positive way

There had to be some major cuts towards the end like the exclusion of proper context management for the npc and a mini boss and boss system for the enemies, but these were necessary cuts to ensure the core parts of my project were completed in time.

Major Technical Achievements

I was able to achieve my 3 main goals, even if not perfectly.

I have a custom C+++ sfml wrapper library that can be attached to any sfml project and work, plug and play. The difficulty in this lied in understanding the proper way to extract tokens from llama at the end of generation and building the tokenised string.

The second was completing the Fully procedurally generated dungeon. The deterministic aspect of it, allowing users to redo the same generated level as well as how scalable it is due to its component-based design.

Lastly is the completion of the llama -> wrapper -> SFML flow with the ability to send and receive tokens from the llm as well as implementing the use cases to test my hypothesis. Lore is generated via the llm, the player can interact with NPC's by sending messages that they type to them, and rooms give a description of what they are.

Project Review

This project has been a huge lesson in proper time management but also a victory in perseverance. Pushing in the hardest of moments to cross the finish line. One of the things I did right was the extensive research I did into this technology at the beginning before touching a single bit of code as this is what saved me time when it came to moving straight to the prototyping phase of building the wrapper and integrating it.

There was supposed to be a mini boss system, inspired from the middle earth shadow of war games where within certain combat rooms, the player would encounter a procedurally generated enemy that generated it's lore off the procedurally generated elements but I had to divert my attention to the finetuning of the llm when I realised I could not have both at once.

One of the things that doesn't sit right with me though is the not being able to crack the proper prompt structure to get the exact results desired when passing a prompt through.

Conclusions

In conclusion, though I could not receive a positive answer to the question I posed at the start of this report, I can conclude that there is still potential for the application of local inference within video games.

Future Work

In terms of improving inference time, tightening the wrapper to be a lot more explicit with the types being returned to help with efficiency would go along way in making inference on all fronts very fast.

Another to be looked at is running the llm on GPU cores instead of CPU cores and measuring the difference.

Finally, setting the kv_cache could help with immersing the player when interacting with NPCs and then also help with their generation.

References

Websites:

Top 20 Applications of Large Language Models in Real-Life (2025). Available at: https://www.geeksforgeeks.org/artificial-intelligence/top-20-applications-of-large-language-models-in-real-life-/#4-entertainment (Accessed April 2026)

Arm Large Language Models Repo (2025). Available at: https://gitlab.arm.com/kleidi/kleidi-examples/large-language-models (Accessed April, 2026)

Arm Real Time Voice Assistant (2025). Available at: https://gitlab.arm.com/kleidi/kleidi-examples/real-time-voice-assistant (Accessed April, 2026)

Large Language Model: A Guide To The Question 'What Is An LLM" (2024). Available at: https://www.eweek.com/artificial-intelligence/large-language-model/ (Accessed April, 2026)

Explaining Tokens — the Language and Currency of AI (2025): Available at: https://blogs.nvidia.com/blog/ai-tokens-explained/ (Accessed April, 2026)

Transformers Part 1: A High-Level Overview Available at: https://medium.com/@navinaananthan/transformers-part-1-a-high-level-overview-de0c8d47f684 (Accessed April,2026)

Lucy's Labrynth (2024) Available at: https://github.com/MorganRO8/Lucys_Labyrinth (Accessed April 2026)

Appendices